

LibroSystemRR — Cronograma de Desarrollo (Semanas 3 y 4)

Estructura de Datos y Algoritmos I · EPN – FIS

Integrantes: Sebastián Ríos · Ariel Recalde

Período: 08/06/2026 → 19/06/2026

Leyenda RACI

Código	Significado	Código	Significado
R	Responsable – ejecuta la tarea	A	Aprobador – revisa y da visto bueno
C	Consultado – aporta criterio técnico	I	Informado – recibe actualizaciones

SEMANA 3 — Iteración 1: TADs y Modelos Base

Período: 08/06/2026 → 12/06/2026 | **Entregable:** Avance Funcional 1

Objetivo: Implementar todos los TADs, las interfaces, las excepciones, el **SistemaLogger** y los modelos base del sistema. Al cierre del viernes, el proyecto debe compilar sin errores y todos los componentes deben tener prueba manual exitosa.

Lunes 08/06/2026 — Estructura del proyecto y TADs lineales base

Actividad	Entregable	Sebastián	Ariel
Crear estructura de carpetas y paquetes en VS Code	Proyecto compilando	R	C
Verificar package en cada archivo Java	Proyecto compilando	R	C
Implementar Nodo<T>	Nodo.java	R	I
Implementar ListaEnlazada<T>	ListaEnlazada.java	R	C
Implementar interfaces Buscable y Ordenable<T>	Buscable.java , Ordenable.java	C	R

Meta del día: El proyecto compila sin errores. **Nodo<T>** y **ListaEnlazada<T>** funcionando con prueba manual básica.

Martes 09/06/2026 — TADs restantes y excepciones

Actividad	Entregable	Sebastián	Ariel
Implementar Cola<T>	Cola.java	R	I
Implementar Pila<T>	Pila.java	R	I
Implementar NodoBST	NodoBST.java	I	R
Implementar CatalogoBST (insertar, buscar rec., inorden)	CatalogoBST.java	C	R
Implementar todas las excepciones personalizadas	excepciones/	R	C

Meta del día: Los 5 TADs implementados y compilando. Excepciones listas. **NodoBST** inserta **Libro** correctamente.

Miércoles 10/06/2026 — Modelos de dominio y SistemaLogger

Actividad	Entregable	Sebastián	Ariel
Implementar Libro con Ordenable<T> y compareTo()	Libro.java	R	C
Implementar clase abstracta Usuario	Usuario.java	R	C
Implementar Lector con validación de límite y deudas	Lector.java	R	C
Implementar SistemaLogger (Singleton)	SistemaLogger.java	I	R

Meta del día: Todos los modelos base implementados. **Libro** insertable en **CatalogoBST**. **SistemaLogger** funcionando.

Jueves 11/06/2026 — Datos de prueba y pruebas manuales

Actividad	Entregable	Sebastián	Ariel
Implementar cargarDatosDePrueba() en Main.java	Datos en memoria al iniciar	R	C
Probar inserción de libros en CatalogoBST	Prueba manual	R	R
Probar inorden() del BST — verificar orden alfabético	Prueba manual	C	R
Probar ListaEnlazada, Cola y Pila con datos reales	Prueba manual	R	C
Probar validaciones de Lector (límite 3, deuda)	Prueba manual	R	C
Probar lanzamiento de excepciones personalizadas	Prueba manual	C	R

Meta del día: Todo el Avance Funcional 1 probado manualmente. Sin errores en consola.

Viernes 12/06/2026 — Entrega y revisión

Actividad	Entregable	Sebastián	Ariel
Revisión final del código — convenciones y Javadoc	Código limpio	R	R
Verificar que ningún TAD usa <code>java.util.*</code>	Código limpio	R	R
Subir avance al repositorio	Repositorio actualizado	R	R
Redactar resumen del Avance Funcional 1	Documento breve	A	R

Distribución de Carga — Semana 3

Día	Sebastián	Ariel
Lunes	Estructura VS Code · <code>Nodo<T></code> · <code>ListaEnlazada<T></code>	Interfaces <code>Buscable</code> · <code>Ordenable<T></code>
Martes	<code>Cola<T></code> · <code>Pila<T></code> · Excepciones	<code>NodoBST</code> · <code>CatalogoBST</code>
Miércoles	<code>Libro</code> · <code>Usuario</code> · <code>Lector</code>	<code>SistemaLogger</code>
Jueves	<code>cargarDatosDePrueba()</code> · Pruebas TADs lineales y modelos	Pruebas BST · Pruebas excepciones
Viernes	Revisión código · Convenciones · Repositorio	Revisión código · Documento avance

Checklist de Entrega — Avance Funcional 1

TADs

- ☐ `Nodo<T>` — dato, siguiente, getters y setters
- ☐ `ListaEnlazada<T>` — agregar, agregarAlInicio, eliminar, obtener, buscar, getTamaño, estaVacia, toArray
- ☐ `Cola<T>` — encolar, desencolar, getFronte, getTamaño, estaVacia
- ☐ `Pila<T>` — apilar, desapilar, getTope, getTamaño, estaVacia
- ☐ `NodoBST` — libro, izquierdo, derecho, getters y setters
- ☐ `CatalogoBST` — insertar, buscarPorIsbn, buscarPorTitulo, eliminar, inorden, estaVacio, getTamaño + métodos recursivos privados

Interfaces y Excepciones

- ☐ `Buscable` — buscarPorIsbn, buscarPorTitulo
- ☐ `Ordenable<T>` — compareTo
- ☐ `LibroSystemException` y sus 5 subclases implementadas

Modelos

- ☐ `Libro` — todos los atributos, compareTo(), isDisponible(), setDisponible()
- ☐ `Usuario` — clase abstracta, prestamosActivos: `Cola<Prestamo>`, historialPrestamos: `Pila<Prestamo>`, getTipo() abstracto

- ☐ **Lector** — puedePrestar(), tieneDeudas(), limitePrestamos = 3
- ☐ **SistemaLogger** — patrón Singleton funcionando, registrarInfo, registrarError, registrarWarning
- ☐ **cargarDatosDePrueba()** con mínimo 5 libros y 3 usuarios

Calidad

- ☐ Ningún archivo importa `java.util.ArrayList`, `java.util.LinkedList` ni `java.util.Stack`
- ☐ Todos los paquetes declarados correctamente (`package librosystemrr.tads;` etc.)
- ☐ Todos los atributos son `private` o `protected`
- ☐ Javadoc en todos los métodos públicos

Riesgos — Semana 3

Riesgo	Probabilidad	Impacto	Mitigación
CatalogoBST recursivo toma más tiempo de lo esperado	Media	Alto	Ariel arranca el lunes revisando NodoBST para no esperar al martes
Errores de paquetes en VS Code	Alta	Medio	Sebastián verifica la estructura el lunes antes de codificar nada
TADs sin probar llegan a Semana 4	Media	Alto	Jueves es día exclusivo de pruebas, no de implementación nueva

SEMANA 4 — Iteración 2: Algoritmos y Préstamos

Período: 15/06/2026 → 19/06/2026 | **Entregable:** Avance Funcional 2

Objetivo: Implementar los algoritmos de búsqueda y ordenamiento, el módulo de préstamos completo (**Prestamo**, **Multa**, **SistemaBiblioteca**), y los modelos **Bibliotecario** y **AyudanteBibliotecario**. Al cierre del viernes, el sistema debe poder registrar préstamos, procesar devoluciones y ordenar el catálogo desde consola.

Lunes 15/06/2026 — Algoritmos de búsqueda

Actividad	Entregable	Sebastián	Ariel
Implementar AlgoritmosBusqueda — <code>busquedaLineal</code>	AlgoritmosBusqueda.java	R	C
Implementar <code>buscarPorTituloLineal</code> y <code>buscarVencidos</code>	AlgoritmosBusqueda.java	R	C
Implementar <code>busquedaBinaria</code> (arreglo ordenado)	AlgoritmosBusqueda.java	R	C
Implementar <code>buscarEnBST</code> (recursivo)	AlgoritmosBusqueda.java	C	R

Meta del día: **AlgoritmosBusqueda** completo y probado. Los 3 tipos de búsqueda funcionan contra datos de prueba de Semana 3.

Martes 16/06/2026 — Algoritmos de ordenamiento y SistemaBiblioteca (base)

Actividad	Entregable	Sebastián	Ariel
Implementar <code>AlgoritmosOrdenamiento</code> — mergeSort (recursivo)	<code>AlgoritmosOrdenamiento.java</code>	C	R
Implementar <code>ordenarPorTitulo</code> y <code>ordenarPorAutor</code>	<code>AlgoritmosOrdenamiento.java</code>	C	R
Implementar quickSort in-place (recursivo)	<code>AlgoritmosOrdenamiento.java</code>	C	R
Implementar <code>ordenarPorFechaDevolucion</code>	<code>AlgoritmosOrdenamiento.java</code>	C	R
Crear esqueleto de <code>SistemaBiblioteca</code> con atributos y constructor	<code>SistemaBiblioteca.java</code>	C	R

Meta del día: Ambos algoritmos de ordenamiento funcionando. `SistemaBiblioteca` compila con sus dependencias inyectadas.

Miércoles 17/06/2026 — Modelos Prestamo, Multa, Bibliotecario y AyudanteBibliotecario

Actividad	Entregable	Sebastián	Ariel
Implementar <code>Prestamo</code> con <code>isVencido()</code> y <code>getDiasRetraso()</code>	<code>Prestamo.java</code>	C	R
Implementar <code>Multa</code> con <code>MONTO_POR_DIA</code> y <code>pagar()</code>	<code>Multa.java</code>	C	R
Implementar <code>Bibliotecario</code> con <code>registrarLibro()</code> y <code>procesarDevolucion()</code>	<code>Bibliotecario.java</code>	R	C
Implementar <code>AyudanteBibliotecario</code> con <code>registrarPrestamo()</code>	<code>AyudanteBibliotecario.java</code>	R	C

Meta del día: `Prestamo` y `Multa` funcionando. `Bibliotecario` y `AyudanteBibliotecario` compilando con `getTipo()` correcto.

Jueves 18/06/2026 — SistemaBiblioteca completo e integración

Actividad	Entregable	Sebastián	Ariel
Implementar <code>registrarLibro()</code> y <code>registrarUsuario()</code>	<code>SistemaBiblioteca.java</code>	C	R
Implementar <code>registrarPrestamo()</code> con validaciones	<code>SistemaBiblioteca.java</code>	C	R
Implementar <code>procesarDevolucion()</code> con cálculo de multa	<code>SistemaBiblioteca.java</code>	C	R
Implementar <code>generarAlertas()</code> y <code>ordenarCatalogo*()</code>	<code>SistemaBiblioteca.java</code>	C	R

Actividad	Entregable	Sebastián	Ariel
Pruebas de integración algoritmos + sistema completo	Prueba manual	R	R

Meta del día: **SistemaBiblioteca** completo. Se puede registrar un préstamo, procesarlo y calcular multas desde **Main**.

Viernes 19/06/2026 — Entrega y revisión

Actividad	Entregable	Sebastián	Ariel
Verificar complejidades anotadas en Javadoc	Código limpio	R	R
Revisión final — convenciones, encapsulamiento	Código limpio	R	R
Subir avance al repositorio	Repositorio actualizado	R	R
Redactar resumen del Avance Funcional 2	Documento breve	R	A

Distribución de Carga — Semana 4

Día	Sebastián	Ariel
Lunes	AlgoritmosBusqueda — búsqueda lineal y binaria	AlgoritmosBusqueda — buscarEnBST recursivo
Martes	Apoyo AlgoritmosOrdenamiento	AlgoritmosOrdenamiento completo · SistemaBiblioteca (base)
Miércoles	Bibliotecario · AyudanteBibliotecario	Prestamo · Multa
Jueves	Pruebas integración algoritmos + modelos	SistemaBiblioteca completo (métodos de negocio)
Viernes	Revisión código · Complejidades Javadoc · Repositorio	Revisión código · Documento avance

Checklist de Entrega — Avance Funcional 2

Algoritmos

- ☐ **AlgoritmosBusqueda** — `busquedaLineal`, `buscarPorTituloLineal`, `buscarVencidos`, `busquedaBinaria`, `buscarEnBST`
- ☐ **AlgoritmosOrdenamiento** — `mergeSort` + recursivos privados, `ordenarPorTitulo`, `ordenarPorAutor`, `ordenarPorFechaDevolucion`
- ☐ **AlgoritmosOrdenamiento** — `quickSort` + `quickSortRec` + `partition` (in-place)

Modelos nuevos

- ☐ **Prestamo** — id, libro, usuario, fechaPrestamo, fechaDevolucion, devuelto, multa, isVencido(), getDiasRetraso(), devolver()
- ☐ **Multa** — monto, pagada, fechaPago, MONTO_POR_DIA = 0.50, pagar()
- ☐ **Bibliotecario** — codigoEmpleado, getTipo(), registrarLibro(), procesarDevolucion(), generarReporte()
- ☐ **AyudanteBibliotecario** — codigoEmpleado, getTipo(), registrarPrestamo(), procesarDevolucion()

Sistema

- ☐ **SistemaBiblioteca** — registrarLibro, buscarLibroPorIsbn, buscarLibroPorTitulo
- ☐ **SistemaBiblioteca** — registrarUsuario, buscarUsuario
- ☐ **SistemaBiblioteca** — registrarPrestamo con validaciones (puedePrestar, tieneDeudas, disponibilidad)
- ☐ **SistemaBiblioteca** — procesarDevolucion con generación automática de Multa si hay retraso
- ☐ **SistemaBiblioteca** — generarAlertas, ordenarCatalogoPorTitulo, ordenarCatalogoPorAutor

Calidad

- ☐ Complejidades documentadas en Javadoc de cada método de algoritmos
- ☐ Métodos recursivos privados en **AlgoritmosBusqueda** y **AlgoritmosOrdenamiento**
- ☐ **SistemaBiblioteca** NO accede directamente a TADs desde la UI — solo desde negocio
- ☐ Todas las excepciones de Semana 3 lanzadas correctamente desde **SistemaBiblioteca**

Riesgos — Semana 4

Riesgo	Probabilidad	Impacto	Mitigación
SistemaBiblioteca concentra demasiada lógica en un día	Alta	Alto	Ariel trabaja el martes ya en el esqueleto para que el jueves solo complete métodos
mergeSort sobre ListaEnlazada es complejo sin acceso aleatorio	Media	Medio	Usar estrategia divide: extraer mitad con puntero lento/rápido (tortuga-liebre)
Prestamo depende de Usuario y Libro — ciclos de compilación	Media	Medio	Verificar que Libro y Lector compilen antes de arrancar Prestamo.java el miércoles
Tiempo insuficiente para pruebas de integración	Media	Alto	Jueves es exclusivo para SistemaBiblioteca + pruebas. No agregar features nuevas